

La **normalización de bases de datos** es un proceso técnico que consiste en organizar los datos de una base de datos relacional. Su objetivo principal es diseñar estructuras de tablas óptimas para **eliminar la redundancia** (datos duplicados) y **evitar anomalías** en las operaciones de inserción, actualización y borrado de datos.

Este proceso fue introducido por Edgar F. Codd y se basa en una serie de reglas llamadas **Formas Normales (FN)**. Para que una base de datos esté en una forma normal superior, primero debe cumplir estrictamente con las formas normales anteriores.

A continuación, se explican las tres primeras formas normales, que son las más utilizadas en el diseño estándar de bases de datos:

1. Primera Forma Normal (1FN): Atomicidad

Para que una tabla cumpla con la 1FN, debe cumplir con dos condiciones básicas:

- **Campos atómicos:** Cada columna debe contener un solo valor indivisible. No se permiten listas, arreglos ni múltiples datos dentro de una misma celda.
- **Sin grupos repetidos:** No debe haber columnas que almacenen información similar para el mismo registro.

Ejemplo de lo que NO se debe hacer (Violación de la 1FN):

Imagina una tabla de Clientes donde guardas los teléfonos en una sola celda separados por comas:

ID_Cliente	Nombre	Teléfonos
1	Carlos Silva	912345678, 987654321

¿Por qué es un problema? Porque si quieres buscar a los clientes que tengan un número específico o actualizar solo uno de los teléfonos, el motor de la base de datos tendrá dificultades para procesar el texto completo.

Solución en 1FN:

Cada teléfono debe ir en un registro separado, o bien, los teléfonos se extraen a una tabla secundaria:

ID_Cliente	Nombre	Teléfono
1	Carlos Silva	912345678
1	Carlos Silva	987654321

2. Segunda Forma Normal (2FN): Dependencia Funcional Completa

Para aplicar la 2FN, la tabla ya debe estar en **1FN**. La regla de la 2FN dice que **todos los atributos que no forman parte de la clave primaria deben depender por completo de la clave primaria completa**, y no solo de una parte de ella.

*Nota: Esta regla aplica principalmente a tablas que tienen una **clave primaria compuesta** (hecha de dos o más columnas).*

Ejemplo de lo que NO se debe hacer (Violación de la 2FN):

Imagina una tabla Detalle_Factura que registra los productos comprados. La clave primaria es compuesta: (ID_Factura, ID_Producto).

ID_Factura (PK)	ID_Producto (PK)	Cantidad	Nombre_Producto
5001	10	3	Laptop Asus
5002	10	1	Laptop Asus

¿Por qué es un problema? El Nombre_Producto ("Laptop Asus") depende únicamente del ID_Producto (10), no le importa en absoluto el ID_Factura. Si el nombre del producto cambia, tendrías que modificarlo en miles de registros de facturas viejas, generando riesgo de inconsistencias.

Solución en 2FN:

Se divide la información en dos tablas diferentes de modo que las dependencias parciales desaparezcan:

- **Tabla Productos:** ID_Producto (PK), Nombre_Producto
- **Tabla Detalle_Factura:** ID_Factura (PK), ID_Producto (FK), Cantidad

3. Tercera Forma Normal (3FN): Eliminación de Dependencias Transitivas

Para que una tabla esté en 3FN, primero debe estar en **2FN**. La regla establece que **ningún atributo que no sea clave puede depender de otro atributo que tampoco sea clave**. En palabras simples: las

descripciones o datos deben depender directa y únicamente de la clave primaria (la llave), de toda la llave y de nada más que de la llave.

Ejemplo de lo que NO se debe hacer (Violación de la 3FN):

Imagina una tabla de Empleados:

ID_Empleado (PK)	Nombre	ID_Departamento	Nombre_Departamento
101	Ana Gómez	D01	Recursos Humanos
102	Pedro Díaz	D01	Recursos Humanos

¿Por qué es un problema? El Nombre_Departamento depende directamente del ID_Departamento, y el ID_Departamento depende del ID_Empleado. Esto es una **dependencia transitiva**. Si todos los empleados de un departamento renuncian y borramos sus registros, perderíamos la información de que el departamento "D01" se llama "Recursos Humanos".

Solución en 3FN:

Se separa el atributo transitivo creando una tabla independiente para el catálogo de departamentos:

- **Tabla Departamentos:** ID_Departamento (PK), Nombre_Departamento
- **Tabla Empleados:** ID_Empleado (PK), Nombre, ID_Departamento (FK)

Ventajas de la Normalización

1. **Integridad de los datos:** Al actualizar un dato (como el correo de un cliente), solo se modifica en un único lugar, evitando que queden datos viejos en el sistema.
2. **Ahorro de espacio:** No se almacena texto repetido de forma innecesaria en miles de filas.
3. **Estructura clara:** Facilita la comprensión del modelo de datos para los desarrolladores.